Field-Level Financial Document Extraction with YOLO26 and Apache Spark: A Reproducible Pipeline, Its Failure Modes, and a Measured Evaluation

Anonymous Authors
Under double-blind review

Abstract. ***Putting a vision model inside a Spark job looks routine and fails quietly. We build a field-level document extraction pipeline in which YOLO26 localizes fields and Tesseract reads each crop, executed with `mapInPandas` so the model loads once per partition. We catalog the integration defects that made a first version wrong, marking which we reproduced by running code, including a Spark image source that returns decoded pixels instead of file bytes and an Arrow failure on Spark 3.5 with JDK 21. On a synthetic form dataset with exact ground truth, a CPU-fine-tuned YOLO26n reaches 98.7% detection precision and 99.3% recall, and 97.1% of text fields are read correctly end to end. Without class-wise suppression the end-to-end head emitted 90 duplicate boxes and precision fell to 75.9%. Full-page Tesseract captured 99.6% of the strings but does not say which field they belong to, so we do not claim an accuracy advantage. A rule-based baseline over full-page OCR, with one regular expression per class, read 99.2% of the fields correctly, so on this regular synthetic data the detector does not improve accuracy. A `mapInPandas` job that loads the model once per partition was 1.7 times faster than a row-wise UDF that builds it on every call, on two CPU cores. A repeat on one GPU in a different host gave the same conclusions (98.3% end to end) and no inference speedup for the nano model. Data are synthetic, and no cluster, multi-GPU, or Textract measurements are reported. All numbers are generated from files in the released repository.***

Index Terms. ***Financial document intelligence, YOLO26, Apache Spark, document field extraction, OCR, reproducibility.***

I. Introduction

Financial institutions ingest checks, identification forms, and statements as images, and downstream systems need to know which string is the account number, which is the date, and which is the amount. Page-level OCR returns text but not that mapping, and its accuracy is not a fixed number: published evaluations report Tesseract anywhere from 47 percent accuracy on Thai vehicle registration certificates [2] to 92.6 percent word recognition on letters [3], with different metrics, document types, resolutions, and preprocessing. A detect-then-read design addresses both problems. An object detector localizes each field, and OCR runs only on the cropped region, so every extracted string arrives already labeled with its field class.

Ultralytics YOLO26 [8] is a natural detector for this role. Its end-to-end head does not need non-maximum suppression at inference, which removes a post-processing pass from the serving path. Running it over millions of documents calls for a distributed engine, and Apache Spark [14] is the usual choice.

The difficulty is that a model call inside a Spark job looks routine and fails quietly. The RAPIDS Accelerator for Apache Spark [15], [16] accelerates Spark's own SQL and DataFrame plan on GPU. It does not accelerate Python code that loads a model inside a user-defined function, so enabling it does not make model inference fast. The first version of this pipeline contained several such defects, some in the model call pattern, some in data access, and some in the toolchain. We found them only after running the code, which is the reason this paper reports a running system and not a design alone.

This paper builds the pipeline, runs it, measures it, and releases it. Its scope is deliberately narrow. All reported accuracy is on a synthetic form dataset whose labels are exact by construction, timing is on a two-core CPU with one replication on a single GPU, and an AWS EMR deployment is described in the repository documentation but not evaluated.

II. Contributions

- A complete open implementation of field detection, field-level OCR, and aggregation as one Spark job, with unit and end-to-end tests. One command reproduces the CPU results, and a notebook repeats the study on a GPU.
- A catalog of integration defects, marked by whether we reproduced them by running code or found them by inspection, and the corrected partition-level inference pattern that resolves them (Section V).
- An evaluation on a dataset with exact ground truth, with one training run per device: detection precision and recall, end-to-end field accuracy against a full-page Tesseract baseline, and a measured comparison of three ways to run the same model inside Spark (Section VII).

III. Related Work

Detection. Single-network detection was introduced by Redmon et al. [5]. YOLOv9 added programmable gradient information [6], and YOLOv10 was the first in the line to remove non-maximum suppression [7]. YOLO26 keeps native end-to-end inference, removes Distribution Focal Loss, and is a recent Ultralytics release [8]. DocLayout-YOLO adapts the family to document layout analysis [9]. We use YOLO26 because it is the maintained production model and its NMS-free head simplifies the serving path. We do not claim it is the best detector for financial fields, and we do not compare against other detectors.

OCR and document understanding. Tesseract [1] remains a common open-source engine, and its accuracy varies widely with document type and preprocessing [2], [3]. Managed services such as Amazon Textract [4] are an obvious comparison, which we do not make because it needs an AWS account and real documents. Layout-aware transformers such as LayoutLMv3 [10] and OCR-free models such as Donut [11] are stronger, heavier alternatives that solve a broader problem. Our baseline is Tesseract alone, which makes the comparison one of pipeline structure, not one against the state of the art.

Datasets. FUNSD [12] and SROIE [13] are the public datasets closest to this domain, and neither fits. FUNSD annotates form entities as question, answer, header, and other. SROIE provides text-line boxes and key-information strings for receipts. Neither provides region boxes for classes such as account number, date, amount, name, and signature. For that reason we evaluate on a synthetic dataset whose boxes and strings are exact, and we say plainly what that does and does not show.

Systems. Spark's resilient distributed datasets [14] underlie its DataFrame engine. Spark exchanges data with Python through Apache Arrow, and `mapInPandas` [17] exposes that path at partition granularity. Spark can schedule GPUs as task resources [18], and the RAPIDS documentation describes how GPU-scheduled Python workers share a device with the executor [15].

IV. System Design

This section describes the Spark job that is implemented and measured in this paper. A design for deploying it on AWS EMR is kept in the repository documentation and is not evaluated here.

A. Spark job

The job reads images with Spark's `binaryFile` source, which returns the encoded file bytes unchanged. A `mapInPandas` function then runs once per partition. It loads the model once, decodes each Arrow batch of images, makes one batched detector call, crops every detected field from the original image, and runs Tesseract in single-line mode on each crop. Signature regions are detected but not read. Output is long format, one row per detected field, written as Parquet. A final step computes per-class mean confidence with a two-phase aggregation: each Arrow batch emits a partial sum and count, and Spark performs the final reduce. That step uses cuDF when RAPIDS is installed and pandas otherwise.

B. Scope of deployment

The design targets batch and micro-batch workloads, such as an overnight queue of scanned documents, and not sub-second per-request latency. A cluster start-up cost is not measured. We ran a GPU only in local mode, by naming the device in an environment variable (Section VII-F). The code can also read the device that Spark assigns to each task under its custom resource scheduling [18] when started with `-gpu`, but that path was not exercised, so it is untested. The AWS EMR design is described in `docs/AWS_DEPLOYMENT.md` and is not evaluated.

V. Implementation and Defects

Table I lists the defects found in the first version of this pipeline. Rows 1 to 4 were reproduced by running code, and the per-row cost in row 4 is measured in Section VII. Rows 5 to 8 were found by inspecting the first version, and row 7 was not exercised under Spark GPU scheduling. Row 8 concerns code from that first version that is not in the released repository.

Listing 1 shows the core of the corrected pattern. The complete module, tests, and a one-command reproduction are in the released repository.[1]

```
def detect_partition(batches):
  device = _task_device()   # GPU from Spark, else CPU
  model = YOLO(SparkFiles.get(name))  # once
  for batch in batches:     # one Arrow batch
    imgs = [cv2.imdecode(
              np.frombuffer(bytes(raw), np.uint8),
              cv2.IMREAD_COLOR)
            for raw in batch["content"]]
    results = model(imgs, device=device)  # 1 call
    for path, im, res in zip(batch["path"],
                             imgs, results):
      for b in res.boxes:   # original pixels
        x1, y1, x2, y2 = map(int,
                             b.xyxy[0].tolist())
        text = pytesseract.image_to_string(
            gray(im[y1:y2, x1:x2]),
            config="--psm 7").strip()
        rows.append((path, cls(b), ..., text))
    yield pd.DataFrame(rows, columns=COLS)
```

Listing 1. Partition-level detection and OCR (abridged from `findoc/pipeline.py`).

VI. Experimental Setup

Data. A deterministic generator renders account forms at 960 by 640 pixels with five field classes: account number, date, amount, name, and signature. All forms share one page layout. Field order, position offsets, the choice among three DejaVu fonts, noise, and blur vary per document. Every box and every text string is known exactly. The split is 240 training, 60 validation, and 60 test documents, with disjoint random seeds. The test split holds 300 fields, of which 240 carry text.

Model. YOLO26n fine-tuned from the COCO checkpoint for 15 epochs at 640 pixels, batch size 8, AdamW, on CPU in about 32 minutes, and once more on a GPU in about 117 seconds. We use the nano model because it is the one

[1] ANONYMOUS-REPO-URL

TABLE I
INTEGRATION DEFECTS FOUND BY RUNNING THE PIPELINE, AND THE FIX APPLIED IN THE RELEASED CODE.

Defect	Observed behavior	Fix
Spark `image` source fed to an image decoder	Returns decoded pixels, not file bytes. A 960 by 640 pixel image gave 1,843,200 bytes against a 132,098 byte JPEG, and `imdecode` returned `None`.	Read with `binaryFile`, which returned bytes identical to the file.
Re-opening a path with `cv2.imread`	An `s3://` URI returns `None`. The path cannot be opened by OpenCV.	Decode the bytes Spark already read; never re-open the path.
`mapInPandas` on Spark 3.5.3 with JDK 21	Fails in the JVM with `sun.misc.Unsafe or DirectByteBuffer.<init>(long, int) not available`, including with `-add-opens` and the Netty reflection flag.	Spark 4.0.1 on JDK 21 ran correctly. Spark 3.5 on JDK 17 is the documented alternative and was not tested.
Model constructed inside a row-wise UDF	The model loads once per row. Cost measured in Section VII.	Load once per partition inside `mapInPandas`.
Per-image Python loop around the model call	Inference is one image at a time.	One batched call per Arrow batch, with a dynamic-batch ONNX export.
Manual resize to 640 by 640	Distorts non-square pages, and every box then needs mapping back to page coordinates.	Pass the original image. Ultralytics letterboxes internally and returns boxes in original pixels.
Hard-coded `device=0` with two concurrent GPU tasks	Two model copies can land on one device. *Not exercised under Spark GPU scheduling.*	Read the device from the task context and schedule one GPU per task.
cuDF used to count a substring in one JSON string	Too small an operation to benefit from a GPU, and the key it searched for did not match the output schema.	Per-partition partial sums and counts by class, reduced by Spark.

that trains on the CPU machine. The released training script defaults to the largest model, and results with it would differ.

Hardware and software. Primary runs: two virtual CPU cores, 7 GB of memory. Replication: one T4 GPU in Google Colab. CPU machine: Python 3.13, Spark 4.0.1, OpenJDK 21, Ultralytics 8.4, ONNX Runtime 1.29, Tesseract 5.3.4. Colab: Spark 4.0.4, ONNX Runtime GPU 1.31, Tesseract 5.3.4. Class-wise suppression (IoU 0.5) before OCR is an option of the pipeline and is off by default.

Metrics. A detection is correct if its class matches and its IoU with an unmatched ground-truth box is at least 0.5. We report precision and recall. End-to-end field accuracy is the share of text fields that are both localized and read with an exact string match after whitespace is removed. The baseline is Tesseract run on the full page. We count a field as captured if its exact string appears anywhere in the page text. That rule is lenient, because the baseline is never required to say which field a string belongs to, so it is an upper bound on what page-level OCR delivers and is not a like-for-like extractor.

Inference-pattern timing. The same fine-tuned model scores 48 test documents on two partitions in three configurations: a row-wise UDF that builds the model on every call, and `mapInPandas` with Arrow batch sizes 1 and 16. Detection only, no OCR, so the comparison isolates the pattern. Each configuration has one discarded warm-up run and 3 timed runs.

Limits of the design. There is one training seed, and the test split is 60 documents, so intervals are wide and we report Wilson 95 percent intervals where they matter. The dataset is synthetic, clean, and easy by comparison with scanned documents, so absolute accuracy here says little about real documents.

VII. RESULTS

All numbers in this section are generated from the result files in the repository. The confidence threshold for each variant was chosen on the validation split by detection F1, and each variant was then scored once on the test split. After the final code changes we reran the evaluation protocol with the released code on the stored CPU model, and all scores, counts and chosen thresholds were identical to the files reported here, so the results are not from older code.

A. Detection and end-to-end accuracy

Table II compares three operating points of the same fine-tuned model, which reaches a validation mAP50 of 0.995 and mAP50-95 of 0.934. Those headline scores say little about deployment behavior. At the default confidence of 0.25, the exported end-to-end head produced 374 boxes for 300 true fields, and precision was only 75.9%. Of those, 90 were duplicates of a field that already had a box, with a mean confidence of 0.59 against 0.72 for the box matched to each true field, and 16 fields had no box above the threshold. The NMS-free head is therefore not duplicate-free after a short nano fine-tune.

Three observations follow. First, adding class-wise suppression before OCR lifts precision from 75.9% to 98.7% and end-to-end accuracy to 97.1%, and it also removes duplicate crops from the OCR stage. Second, tuning an NMS-free detector for validation F1 chose a high threshold (0.45) that traded away recall, and end-to-end accuracy fell to 84.6%, below the untuned default. F1 is not the end-to-end objective, and a threshold chosen for one can hurt the other. Third, the OCR stage is rarely the bottleneck. Among correctly localized text fields it was exact 97.9% of the time with class NMS,

TABLE II

TEST-SPLIT RESULTS, 60 DOCUMENTS AND 300 FIELDS (240 WITH TEXT). P AND R ARE DETECTION PRECISION AND RECALL AT IOU 0.5. OCR IS EXACT MATCH AMONG LOCALIZED TEXT FIELDS. P AND R COUNT ALL FIVE CLASSES, INCLUDING THE 60 SIGNATURES THAT ARE DETECTED BUT NOT READ. END-TO-END (E2E) IS THE SHARE OF THE 240 TEXT FIELDS BOTH LOCALIZED AND READ EXACTLY, WITH ITS WILSON 95% INTERVAL.

Variant (conf)	P	R	OCR	E2E	95% interval
NMS-free (0.25, default)	75.9	94.7	98.7	92.1	88.0 to 94.9
NMS-free (0.45, tuned)	88.8	87.0	100.0	84.6	79.5 to 88.6
Class NMS (0.15, tuned)	98.7	99.3	97.9	97.1	94.1 to 98.6

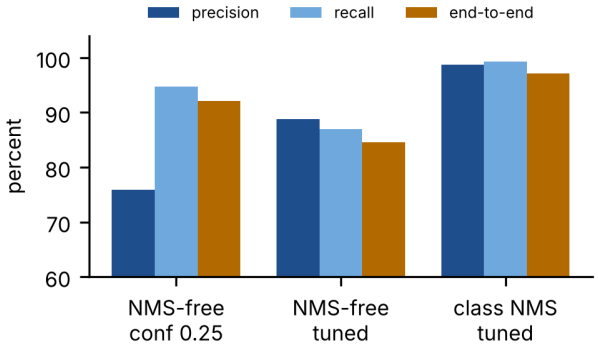


Fig. 1. Precision, recall, and end-to-end accuracy for the three operating points in Table II. All values are percentages on one axis.

and all 5 errors were on account numbers: `8302653675` read as `B302653675`; `4077916523` read as `1077916523`; `0188416938` read as `018841693`.

Fig. 2 gives the per-class breakdown for the class-NMS variant. Account number is the weakest class, in both recall (96.7%) and OCR (91.4%). Class-wise suppression removed same-class duplicates but not every extra box: 4 extra boxes remained on the CPU run and 1 on the GPU run, and none overlaps a true field of its own class at IoU 0.5, so the duplicate count does not include them. The OCR step reads the exact detected box in single-line mode, with no padding, upscaling or thresholding. Several errors look like crop effects, such as a stray leading letter or a dropped final digit, so we ablated the crop handling on the validation split (Section VII-D).

B. Comparison with full-page Tesseract

Full-page Tesseract found 99.6% of the 240 text strings somewhere on the page (95% interval 97.7 to 99.9), which is higher than the 97.1% end-to-end figure. We do not read this as a win for either side. On clean, large, printed text, page-level OCR reads almost every string, and our matching rule is lenient to it, because it never has to say which field a string belongs to. The detect-then-read pipeline pays for field attribution, which the baseline does not provide at all, and on this dataset it does not buy better reading. Whether detection helps on degraded scans, where page-level OCR loses accuracy [2], [3], is the question this dataset cannot answer. Baseline cost was 0.63 seconds per document on one process.

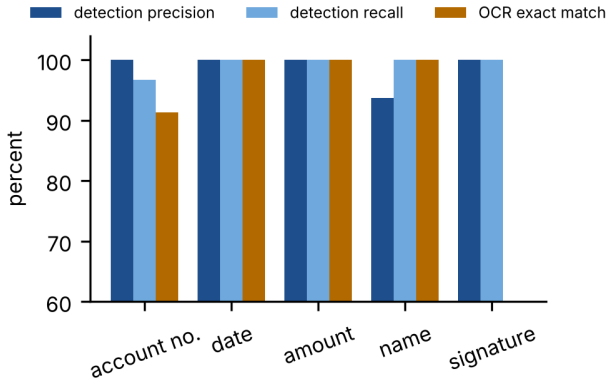


Fig. 2. Per-class detection precision, recall, and OCR exact match for the class-NMS variant on the test split. The vertical axis starts at 60 percent. Signatures are detected but not read.

TABLE III

OCR CROP ABLATION ON THE VALIDATION SPLIT (240 TEXT FIELDS). END-TO-END ACCURACY AND NUMBER OF MISREAD FIELDS.

Crop preparation	E2E, %	Misread
As reported (no padding)	97.5	3
Padding 4 px	97.1	4
Padding 8 px	97.9	2
Padding 4 px, upscale 2x	93.3	13
Padding 4 px, upscale 2x, Otsu threshold	98.3	1

C. Rule-based baseline

A reviewer would ask how much the detector adds on this data, so we also ran full-page Tesseract followed by one regular expression per class (ten digits, a date, a dollar amount, two capitalized words), taking the first match on the page. The patterns come from the formats the generator produces, which makes this baseline generous. It read 99.2% of the 240 text fields correctly (95% interval 97.0 to 99.8), against 97.1% for the detector pipeline, at 0.87 seconds per document on one process. On this synthetic data the rule-based baseline is therefore at least as accurate as the detector pipeline, and the detector gives no accuracy benefit here. We do not expect the same on real forms, where layouts vary and strings are not this regular, but this experiment cannot show it.

D. OCR crop handling

Table III re-reads the same class-NMS detections on the validation split with different crop preparation. The headline test results use the first row and were not changed after this ablation.

Accuracy ranged from 93.3% to 98.3%, and the spread between most rows is a handful of fields. Upscaling alone made reading worse, and adding a threshold recovered it, so preprocessing matters, but no setting is clearly better than the plain crop on 240 fields. We therefore keep the plain crop and do not read the few remaining errors as a limit of Tesseract itself.

TABLE IV
INFERENCE PATTERN TIMING, MEAN AND STANDARD DEVIATION OF 3 RUNS AFTER ONE WARM-UP. CPU ONLY, TWO CORES.

Pattern	Seconds	Docs/s	Speedup
Row-wise UDF, model per row	9.4 ± 0.4	5.1	1.0
`mapInPandas`, batch 1	5.7 ± 0.1	8.5	1.7
`mapInPandas`, batch 16	5.6 ± 0.1	8.6	1.7

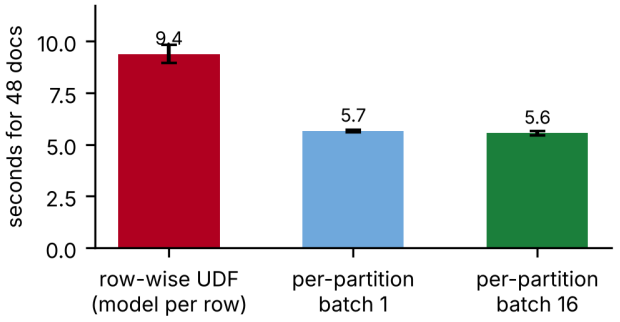


Fig. 3. Wall time for 48 documents under three inference patterns. Error bars are one standard deviation over 3 runs.

E. Inference pattern

Table IV and Fig. 3 compare three ways of running the same detector over 48 documents in Spark on two partitions, with detection only. All three produced the same 300 boxes, so they compute the same thing.

Loading the model once per partition was 1.7 times faster than loading it per row. The extra cost per row of the row-wise pattern is about 167 ms. That is the whole difference between the two patterns, so it includes building the model and also serializing each image to a separate Python call, and we did not separate the two. On the CPU, Arrow batch size 16 was not distinguishable from batch size 1 (5.6 against 5.7 seconds, within one standard deviation). On the GPU, batch 16 was slightly slower than batch 1 (7.3 against 7.0 seconds), so batching did not help on either device for this nano model.

F. Replication on a GPU

We repeated the whole protocol once on a single NVIDIA T4 GPU in Google Colab, with the same repository code, training settings and Tesseract version. The host differed (different CPU, Spark 4.0.4, the GPU build of ONNX Runtime, and a final aggregation that used cuDF), so differences in timing between the columns reflect the host as well as the device, and the Colab CPU model and Java version were not recorded. A recorded environment file shows that ONNX Runtime used the CUDA provider and that the identical test images were produced (SHA-256 prefix 225dd787), so Table V compares the same documents. Training and the confidence sweep were run again, so the weights and the chosen thresholds differ from the CPU run. Spark still ran in local mode with two partitions, so this is a device check and does not exercise Spark GPU scheduling or the RAPIDS Accelerator.

TABLE V
SAME PROTOCOL ON CPU (TWO CORES) AND ON ONE T4 GPU. BOTH ARE SINGLE RUNS ON THE SAME 60 TEST DOCUMENTS.

	CPU	T4
Training time	32 min	117 s
Val. mAP50 / mAP50-95	0.995 / 0.934	0.995 / 0.928
Class NMS: P / R, %	98.7 / 99.3	99.7 / 100.0
Class NMS: end to end, %	97.1	98.3
NMS-free, tuned: end to end, %	84.6	78.8
NMS-free, conf 0.25: end to end, %	92.1	91.3
Duplicates at conf 0.25	90	107
Row-wise UDF, s	9.4	15.4
`mapInPandas`, batch 16, s	5.6	7.3
Speedup, batch 16	1.7	2.1

Detection and reading replicate: class-wise suppression again removed all same-class duplicates and gave 98.3% end-to-end accuracy (95% interval 95.8 to 99.4), and without it the end-to-end head again emitted duplicates (107 at confidence 0.25). The ordering of the three variants is the same on both devices. Differences of a few points between the two columns cannot be separated into device effects and training variance, because each column is one training run on a 60-document test set, and we did not test them for significance. The per-partition speedup over the row-wise pattern held (2.1 times), but the GPU did not make this job faster. Every pattern was slower in the GPU run than in the CPU run, for example 7.3 against 5.6 seconds at batch 16, and the row-wise gap (15.4 against 9.4 seconds) is consistent with an extra CUDA initialization in every call. Host differences are a second possible cause, and we did not profile either. Training was the exception, and was much faster on the GPU. The GPU row therefore supports the same conclusions as the CPU run and shows no inference benefit for this small model.

Two earlier runs on the same GPU are worth recording. Both gave poor reading accuracy (71.7% end to end in the one we kept) because Colab lacked the DejaVu fonts, the generator silently used a different font, and the test images differed. The environment file recorded after the second run showed this, and the only change before the third run was installing the fonts, with no change to code or thresholds. The test split was therefore scored in three Colab runs, and we report the third because it is the first on the same images as the CPU run. Detection was unaffected (recall 100.0%), but reading was not. The released generator now stops with an error in that case. Reading accuracy on synthetic text is sensitive to the rendering environment, which is a further reason to treat all numbers here as properties of this dataset.

G. Dataflow

Fig. 4 contrasts the two inference patterns that Table IV measures.

VIII. DISCUSSION AND LIMITATIONS

What this supports. The pipeline runs end to end under Spark, produces field-labeled text, and its CPU results can

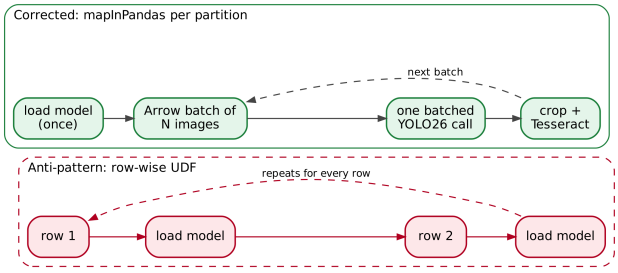


Fig. 4. Row-wise UDF versus partition-level `mapInPandas`. The first reloads the model for every row. The second loads it once per partition and then scores Arrow batches.

be regenerated from one repository. Loading the model per partition is faster than per row, and detection quality at a fixed threshold can differ sharply from the threshold-free mAP that training reports.

What it does not support. The dataset is synthetic, clean, and generated by one randomized form generator with three fonts and varied layout, noise and blur, so it is easy, and the high validation mAP reflects that. The results do not show that detection beats full-page OCR on real documents, and on this data it did not. There is one training run per device and a 60-document test split, so intervals are wide. The detector is the nano model. We ran the protocol on CPU and once on a single GPU in local Spark mode. Multi-node scaling, cluster start-up, cost, Spark GPU scheduling with the RAPIDS Accelerator, and Amazon Textract were not measured, and the cluster GPU configuration in the released code is untested. The rule-based baseline reads these strongly formatted fields at least as accurately as the detector pipeline on this synthetic data, so the dataset gives no accuracy reason to prefer detection over such a rule. The test split was scored for three variants and, for the GPU, in more than one Colab run, although the threshold for each variant was chosen on validation only. The validation split also selected the checkpoint and gives the reported mAP. Stronger baselines, such as LayoutLMv3 [10], Donut [11], and other open OCR engines, were not run.

Cold start. The batch design assumes cluster start-up cost is amortized over a queue of documents. It is a poor fit for sub-second interactive requests, for which a warm cluster or a dedicated model server is the better choice.

Next steps. The most informative next experiment is the same protocol on real, degraded scans with field-level labels, where the comparison with page-level OCR could go either way, followed by the GPU path on EMR with a larger model, where device use may matter more than it did here.

IX. Ethical Considerations

Checks and identification documents contain sensitive personal data. All data here is synthetic. Any use of real documents should follow the institution's data-governance process and data-minimization principles such as those in the GDPR [19], with encryption at rest, separate access scopes for raw images and structured output, and human review for low-confidence detections rather than automated decisions. Synthetic data from one generator does not establish fair performance across banks, regions, and languages, and that should be tested directly.

X. Conclusion

We built and released a Spark pipeline that detects financial form fields with YOLO26 and reads each with Tesseract, and we checked it by running it. The integration defects were real and are cataloged. On clean synthetic forms the pipeline reaches 97.1% end-to-end field accuracy with class-wise suppression, the end-to-end head emitted duplicates without it, and full-page OCR read the same strings at least as well, without saying which field each belonged to. A rule-based baseline with one regular expression per class read 99.2% of the fields on this synthetic data, so the data does not show a benefit from detection. A `mapInPandas` job that loads the model once per partition was 1.7 times faster than a row-wise UDF on CPU, and 2.1 times in a GPU run on a different host, where the nano model gained no inference speed. Real scanned documents, cluster-scale GPU execution, and stronger baselines are the open questions, and the released code is meant to make those experiments cheap to run.

References

[1] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 2, 2007, pp. 629 to 633.

[2] K. Thammarak, P. Kongkla, Y. Sirisathitkul, and S. Intakosum, "Comparative analysis of Tesseract and Google Cloud Vision for Thai vehicle registration certificate," *Int. J. Electrical and Computer Engineering*, vol. 12, no. 2, pp. 1849 to 1858, Apr. 2022.

[3] C. Reisswig, A. R. Katti, M. Spinaci, and J. Höhne, "Chargrid-OCR: end-to-end trainable optical character recognition for printed documents using instance segmentation," arXiv:1909.04469, 2019.

[4] Amazon Web Services, "Amazon Textract." [Online]. Available: https://aws.amazon.com/textract/

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779 to 788.

[6] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: learning what you want to learn using programmable gradient information," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.

[7] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: real-time end-to-end object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: unified real-time end-to-end vision models," arXiv:2606.03748, 2026.

[9] Z. Zhao, H. Kang, B. Wang, and C. He, "DocLayout-YOLO: enhancing document layout analysis through diverse synthetic data and global-to-local adaptive perception," arXiv:2410.12628, 2024.

[10] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: pre-training for document AI with unified text and image masking," in *Proc. ACM Int. Conf. Multimedia*, 2022.

[11] G. Kim *et al.*, "OCR-free document understanding transformer," in *Proc. European Conf. Computer Vision (ECCV)*, 2022.

[12] G. Jaume, H. K. Ekenel, and J.-P. Thiran, "FUNSD: a dataset for form understanding in noisy scanned documents," in *Proc. Int. Conf. Document Analysis and Recognition Workshops (ICDARW)*, 2019.

[13] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, "ICDAR2019 competition on scanned receipt OCR and information extraction," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2019.

[14] M. Zaharia *et al.*, "Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15 to 28.

[15] NVIDIA, "RAPIDS accelerated user-defined functions," RAPIDS Accelerator for Apache Spark documentation. [Online]. Available: https://nvidia.github.io/spark-rapids/docs/additional-functionality/rapids-udfs.html

[16] NVIDIA, "RAPIDS Accelerator for Apache Spark," GitHub repository. [Online]. Available: https://github.com/NVIDIA/spark-rapids

[17] Apache Software Foundation, "pyspark.sql.DataFrame.mapInPandas," Apache Spark documentation. [Online]. Available: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.mapInPandas.html

[18] Apache Software Foundation, "Custom resource scheduling and configuration overview," Apache Spark configuration documentation. [Online]. Available: https://spark.apache.org/docs/latest/configuration.html

[19] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, 2016.